NFC Card Battler — Game Spec & Brainstorm

Oct 16, 2026 · @Reagan

Summary

A terminal roguelite written in C#, where each NTAG215 card holds one character and is that character's only save file. You fight NPC enemies run by utility AI.

Area	Decision
Game	Turn-based roguelite in the terminal; NPC fights only
Platform	C# on .NET 10; Windows PC with the ACS ACR1552U reader
Cards	NTAG215; one character per card for life; the card is the only save (64-byte save, two slots, CRC-32)
NFC stack	PCSC (pcsc-sharp) + Microsoft's Iot.Device.Bindings `UltralightCard`, joined by a custom `PcscTransceiver`; h4kbas/nfc-reader dropped
Architecture	`Core` (battle engine, AI, rules) · `Cards` (`ICardStore`) · `Terminal` front-end · `Tests`
Enemy AI	Utility AI for every enemy; a state machine around it for staged fights such as bosses; Q-learning later
Terminal UI	Spectre.Console
Game data	JSON files; 20 moves at launch
Not in v1	PvP, anti-cheat, phones, GUI
Next step	Course of action, step 1: set up the .NET solution

Contents

1. Decisions: the eight starred choices

2. The concept: core loop and game types considered
3. Card data: save layout, save safety, character lifecycle, example character
4. NPC simulation: approaches table, chosen design, utility AI sketch
5. Technologies: choosing the NFC library, how the pieces fit, other libraries
6. Architecture: diagram of the layers
7. Future: GUI and phone/web: options for later
8. Course of action: the six build steps

★ Decisions

#	Decision	Detail
★ 1	**Game type: turn-based roguelite**	Tap in, fight a run of NPC enemies, tap out to save
★ 2	**New characters start on blank cards**	Character creation only works on a blank card; it never overwrites an existing save
★ 3	**The card is the only save**	No PC-side copy or database
★ 4	**NPC fights only**	No PvP in scope
★ 5	**One character per card, for life**	Only an admin script can wipe a card so a new character can be made
★ 6	**20 moves at launch**	The 128-bit unlock bitmap leaves room to grow
★ 7	**No anti-cheat**	Out of scope; saves carry only a checksum to catch a card pulled off mid-write
★ 8	**Enemy AI: utility AI, plus a state machine for staged fights**	Bosses (and any enemy that needs stages) run utility AI inside a state machine; Q-learning is a later experiment

The concept

Decided: a **turn-based roguelite**. You tap a character card to load your hero, fight a run of NPC enemies in the terminal, then tap the card again to save the XP, cash and moves you earned. The card is the save file, so a character's progress travels with the physical card to any PC with a reader.

Core loop

1. **Tap in.** The game reads the card. A blank card goes to character creation; a card with a save shows its character sheet.
2. **Pick an activity.** A dungeon run (a chain of 3–5 fights with a boss), a single arena fight, or the shop.
3. **Fight.** Turn-based: each turn you pick one of your unlocked moves; the enemy's AI picks its own.
4. **Earn.** Wins give XP and cash; levelling raises stats; cash buys moves in the shop.
5. **Tap out.** The game writes the new state back to the card and reads it back to confirm the save.

Game types to choose from

Type	How a fight works	Strength	Weakness
★ **Turn-based roguelite (chosen)**	Pick a move each turn; runs of fights with a boss	Simple to build in a terminal; long-term progression fits XP on the card	Needs enemy variety to stay fresh
Pokémon-style battler	Turn-based, with types and weaknesses (fire beats nature, and so on)	Familiar; types make move choice matter	Balancing a type chart takes time
Auto-battler	Fights play out on their own; you choose loadout and tactics beforehand	Pure NPC-vs-NPC simulation, fun to watch; PvP is easy	Less moment-to-moment control

Type	How a fight works	Strength	Weakness
Card-duel (deck builder)	Moves act as cards drawn from a deck each turn	Deep strategy; unlocked moves become your deck	Most rules to design
Arena PvP	Two cards tap in one after the other; both players choose moves on one keyboard	Social; good with friends	A reader sees one card at a time, so it is hot-seat only

The other types stay here for reference. Fights are against NPCs only, so the Arena PvP row is out of scope.

Card data

Each card is an **NTAG215** (504 bytes of user memory in pages 4–129, 4 bytes per page), read and written only by the PC reader in v1. A character save is 64 bytes, stored twice for safety. It sits inside a standard **NDEF** message; that costs nothing now and keeps the door open for phones later. Fixed game data (move definitions, enemy stats, class base stats) lives in the game as JSON; the card stores only IDs and numbers.

What is on the card

The NDEF message holds one **save record**: a MIME record of type `application/x-nfcbattler` whose payload is two 64-byte save slots, A and B.

Total use is about 160 of 504 bytes, which leaves room to grow. Pad the record so each slot starts on a page boundary; then saving one slot rewrites only its own 16 pages and never touches the other slot or the record header.

Save slot layout (64 bytes = 16 pages)

Bytes	Contents
0–1	Magic `NB` (marks a game save; a card without it is blank)
2	Format version (start at 1)
3	Class ID (see the class table in the game's JSON)

Bytes	Contents
4	Level
5–8	XP (uint32)
9–12	Cash (uint32)
13–14	Save counter (uint16, +1 every save)
15	Unspent stat points
16–27	HP, ATK, DEF, SPD, MAG, LCK (uint16 each)
28–31	Equipped moves: 4 move IDs, 1 byte each
32–47	Move unlock bitmap: 128 bits, bit *n* = 1 means move *n* is unlocked (20 used at launch)
48–59	Character name, ASCII, up to 12 characters, padded with zeros
60–63	CRC-32 of bytes 0–59, to spot a half-written slot

Making saves safe

- **Two save slots.** Each save goes to the older slot. On load, use the slot with a matching CRC and the higher save counter. A card pulled off mid-write then loses one save, not the character.
- **Read-back check.** After writing, read the pages again and compare before saying "Saved".
- **No anti-cheat.** The CRC only catches damaged writes; anyone with a reader could still edit a save, and that is accepted for this project.

Character lifecycle

1. **Blank card taps in.** No `NB` magic in the save record (or no NDEF at all), so the game offers character creation: pick a class and a name.
2. **Create.** The game formats the card for NDEF if needed, writes the new character into both slots, and reads it back.
3. **Play.** From now on the card always loads that character. The game has no "delete" or "new character" option for a card that already has a save.

4. **Wipe (admin only).** A separate admin script clears the save record so the card is blank again. This is the only way to end a character.

Example character

Ember, Pyromancer, level 7 (1,240 / 1,500 XP), 85 cash, 2 unspent stat points.

Stat	HP	ATK	DEF	SPD	MAG	LCK
Value	64	8	9	12	16	5

ID	Move	Effect	Unlocked
0	Strike	100% ATK physical damage	1
1	Guard	Halve damage taken next turn	1
2	Fireball	140% MAG fire damage	1
3	Ignite	Burn: 5% max HP per turn for 3 turns	1
4	Heat Shield	Reflect 30% of the next hit	0 (shop, 150 cash)
5	Meteor	250% MAG, one turn to charge	0 (shop, 400 cash, level 10)

Ember's move bitmap starts with the byte `0x0F` (moves 0–3 unlocked), and the equipped moves are `00 01 02 03`.

NPC simulation

Decided: every enemy uses **utility AI**, and bosses (or any enemy that needs stages) run it inside a **state machine**. Q-learning is a later experiment. The table keeps all seven options for reference; the chosen ones are starred.

Approaches, simplest first

Approach	How it works	Good for	Effort
Random	Pick any unlocked move	First playable build only	Minutes

Approach	How it works	Good for	Effort
Scripted pattern	Fixed rotation, e.g. charge, charge, slam; the game warns the player one turn ahead	Bosses with readable patterns players learn	Low
Rule-weighted random	Rules shift the odds: below 30% HP, healing weight triples	Common enemies	Low
★ **Utility AI**	Score every move (damage, kill chance, healing, status value) and pick the best, with a little noise	Most enemies; personalities via weights	Medium
★ State machine	States such as Aggressive → Defensive → Desperate, switched by HP or turn count	Bosses with phases	Medium
Expectimax search	Simulate 1–2 turns ahead for each move, averaging over random outcomes, and pick the best result	Final bosses, "hard mode"	High
★ Learned (Q-learning, self-play)	Train an agent by running thousands of simulated fights	A later experiment	Very high

★ Chosen design: utility AI inside a state machine

Every enemy has a state machine; a simple enemy just has one state. Each state carries its own **personality weights** and **move list**, and triggers (HP thresholds, turn counts) switch states. Utility AI then picks the move inside whatever state is current. So every enemy runs the same code path, and a boss is simply an enemy with more states.

```
class Enemy
{
    public StateMachine Phases;   // one state for simple enemies

    public Move ChooseMove(Fight fight)
    {
        Phases.Update(fight);     // check triggers, maybe switch state
        var state = Phases.Current;
        return UtilityAi.ChooseMove(this, fight.Player,
            state.Weights, state.Moves, fight.Rng);
    }
}
```

Example: a Marsh Dragon boss with three stages, as data:

State	Entered when	Weights	Moves
Aloof	Fight starts	Aggression 0.8, Caution 0.5	Claw, Taunt
Enraged	HP ⩽ 60%	Aggression 1.5, Finisher 1.0	Claw, Fire Breath
Desperate	HP ⩽ 20%	Aggression 2.0, Caution 0	Fire Breath, Cataclysm (one-turn charge)

States, triggers and weights can all live in the enemy's JSON file, so a new boss needs no new code.

Build order:

1. Random moves, just to test the battle loop.
2. Utility AI with a single state for every enemy.
3. State machines for bosses.
4. Headless simulator to tune weights.
5. Q-learning experiment, trained with the simulator.

Utility AI sketch

```
Move ChooseMove(Fighter self, Fighter foe, Personality p, Random rng)
{
    return self.UnlockedMoves
        .Select(m => (m, score:
              p.Aggression * ExpectedDamage(m, self, foe) / foe.Hp
            + p.Finisher   * (CanKill(m, self, foe) ? 1 : 0)
            + p.Caution    * HealValue(m, self) * (1 - self.HpFraction)
            + p.Cunning    * StatusValue(m, foe)
            + rng.NextDouble() * p.Chaos))
        .MaxBy(x => x.score).m;
}
```

Personalities are just weight sets: a **Brute** has high Aggression, a **Medic** high Caution, a **Trickster** high Cunning and Chaos.

Ideas that make the NPC side richer

- **Seeded randomness.** Pass one `Random(seed)` through each fight so a fight can be replayed exactly, which makes bugs reproducible and tests deterministic.
- **Headless simulation mode.** Run 10,000 NPC-vs-NPC fights with no UI and print win rates per class and move. This is how you balance the game without playtesting every change.
- **Procedural enemies.** Enemy = archetype (Brute, Medic, Trickster) × element × level scaling, so a small JSON file produces hundreds of enemies.
- **Fights as an event log.** The battle engine emits events (`MoveUsed` , `DamageDealt` , `StatusApplied`) and the UI only renders them. The same engine then drives the terminal, a GUI, a web page or the headless simulator.

Technologies

Stack: C# on .NET 10, **Spectre.Console** for the terminal look, **PCSC** (pcsc-sharp) to talk to the reader, and Microsoft's **Iot.Device.Bindings** for NTAG215 and NDEF. h4kbas/nfc-reader is dropped, because its block reads and writes only work on MIFARE Classic cards.

Choosing the NFC library

Recommended: **PCSC** as the transport plus **dotnet/iot's `UltralightCard`** for the card logic, joined by a small `PcscTransceiver` class you write (about 50 lines). `UltralightCard` already detects NTAG215 and reads and writes NDEF messages, so you do not hand-build page layouts.

Library	What it gives you	NTAG215?	Status	Verdict
Iot.Device.Bindings 4.2.0 (dotnet/iot)	`UltralightCard`: detects NTAG215, page read/write, NDEF read/write/format, originality signature, counters, password; `Iot.Device.Ndef` record classes (URI, text, media)	Yes, by name	Microsoft, MIT, Mar 2026, targets .NET 8 (works from .NET 10)	**Use for card logic**
PCSC 7.0.1 (pcsc-sharp)	Talks to any PC/SC reader; card inserted/removed monitoring	Transport only	BSD-2, Jun 2025, about one release a year	**Use as transport**
h4kbas/nfc-reader 2.0.0	Async events, UID, 16-byte block read/write	No: MIFARE Classic only	MIT, Aug 2026	Drop
NdefLibrary 4.1.0	NDEF encoding only	n/a	Last release 2017	Skip; dotnet/iot covers NDEF
CardWerk SmartCard modules	Full NTAG/Ultralight support over PC/SC	Yes	Commercial licence	Only if paying is fine
FrApp42.ACR122U	Card events and UID	No writes	Small project	Skip

How the pieces fit

`UltralightCard` sends raw NTAG commands to an abstract `CardTransceiver` through `Transceive(target, command, response, protocol)`. Your `PcscTransceiver` implements that with PCSC:

- **READ `0x30`** → `FF B0 00 <page> 10`, and **WRITE `0xA2`** → `FF D6 00 <page> 04 <4 bytes>`. These are the commands your Python scripts already use, so they are known to work on your reader.
- **Everything else** (GET_VERSION `0x60`, which the `UltralightCard` constructor sends first, plus READ_SIG `0x3C` and FAST_READ `0x3A`) goes through the ACR1552U's pass-through command. It is documented in the reference manual v1.09, sections 5.3.4–5.3.5.
- **Test GET_VERSION first.** One user report says this reader can drop the last 3 response bytes from Type 2 tags in pass-through mode.

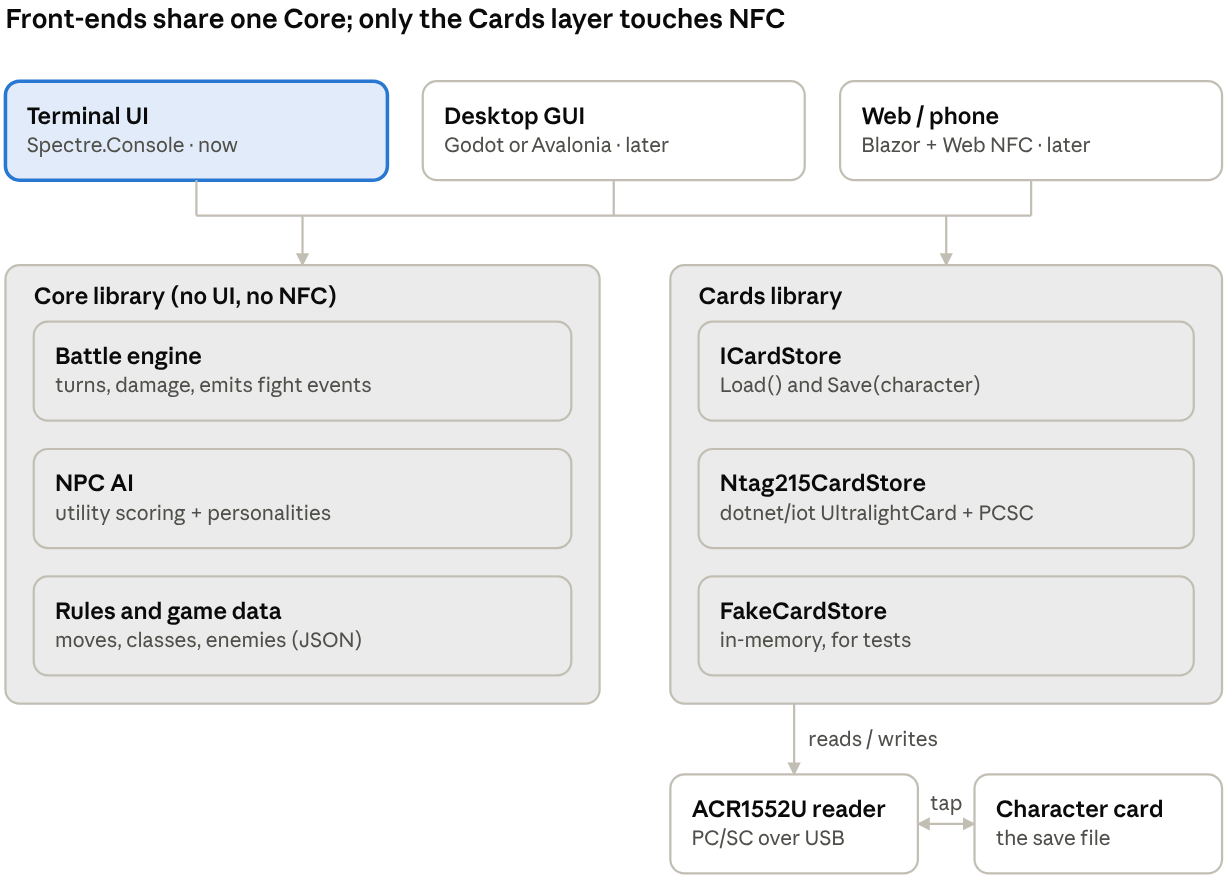
Fallback: if pass-through misbehaves, keep PCSC and write the NDEF bytes yourself; `nfc_write.py` already builds NDEF, and a C# port is about 60 lines. Either way, hide the card behind your own `ICardStore` interface (`Load()`, `Save(character)`) so the game never sees pages or APDUs. The same interface gives you a fake in-memory card for tests and a Web NFC version later.

Libraries and tools

Need	Choice	Notes
Terminal look	Spectre.Console	Coloured panels, tables, HP bars, menus, big ASCII titles, live-updating screens. Best fit for a battle screen.
Full windowed TUI (optional)	Terminal.Gui	Real windows, buttons and mouse support; more work, closer to a GUI.
NFC	Iot.Device.Bindings + `PCSC`	See "Choosing the NFC library"
Game data	`System.Text.Json`	Moves, classes and enemies as JSON files you edit without recompiling
Save checksum	`System.IO.Hashing.Crc32`	Microsoft NuGet package; catches half-written saves

Need	Choice	Notes
Tests	xUnit	Battle engine and card encoding are pure code, testable without a reader
Logging	`Microsoft.Extensions.Logging`	Log card reads and writes while debugging, via `ILogger`

Architecture



project architecture · 3 layers

The Core library has no UI and no NFC code, so every front-end reuses it unchanged. Switching chips or NFC libraries only touches the Cards layer.

Future: GUI and phone/web

Because each save is an NDEF record on an NTAG215, phones can read the same cards without changes: Android Chrome through Web NFC, and iPhones through a native app.

None of this is needed for v1. When phones are added, put a URI record in front of the save record, so a tap opens the game's web page with no app installed.

GUI options

Option	What it is	Fit
Spectre.Console → Terminal.Gui	Fancier terminal first	Smallest step
Godot 4 with C#	Game engine with C# scripting	Best for animation, sprites, sound; reuse the battle engine DLL
MonoGame	Code-first 2D framework	Full control, more manual work
Avalonia	Cross-platform desktop UI	Good for menus and character sheets, less for animation

Phone and web options

Option	How it works	Limits
Web NFC + Blazor web app	Chrome on Android reads the card's NDEF save in the browser; the battle engine runs in a Blazor WebAssembly page	Android Chrome only; no iPhone support in Safari
Card holds an ID, server holds the save	Card stores a URL like `https://<host>/c/<cardId>`; any phone opens it, and the save lives in a database	Works on every phone; needs a hosted server and accounts; the card is no longer the save file
Native app (.NET MAUI)	Android NFC and iOS Core NFC from C#	Full NFC on both platforms; app-store builds needed

Option	How it works	Limits
Hybrid	Save on the card *and* synced to a server on each tap-out	Best of both; needs conflict handling when the card and server disagree

What makes all of these cheap later is keeping the game in layers: a **Core** library (battle engine, NPC AI, rules, no UI), a **Cards** library (`ICardStore` implementations), and thin front-ends (terminal now; Godot, Blazor or MAUI later).

Course of action

Six steps, in order; each ends with something you can run.

1. **Set up C# / .NET.** Create a .NET 10 solution with four projects: `Core` (battle, AI, rules), `Cards` (NFC and save format), `Terminal` (the game) and `Tests` (xUnit).
   - *Done when:* the solution builds and an empty test passes.
2. **Get the NFC library working by replicating the Python scripts.** Add PCSC and Iot.Device.Bindings, write `PcscTransceiver`, then port read, write, clear and backup.
   - *Done when:* the C# output matches `last_read.txt` and the `backup_*.txt` dumps for the same tags. If GET_VERSION fails through pass-through, switch to the PCSC-only fallback here.
3. **Design the character system and card load/save.** Character model, class and move JSON (20 moves), the 64-byte save slot with CRC, two slots, blank-card creation and the admin wipe script, all behind `ICardStore`.
   - *Done when:* you can create a character on a blank card, pull the card, tap it again and get the same character back; a fake card passes the same tests.
4. **Design the battle simulation.** Turn engine that emits events, utility AI, state machines for staged enemies, seeded randomness and the headless simulator.
   - *Done when:* the simulator runs 1,000 fights and prints win rates.
5. **Design the game system.** Roguelite runs (3–5 fights and a boss), XP and levelling, cash and the move shop, and tap-out saving.
   - *Done when:* a full run plays start to finish with plain console text and the rewards land on the card.
6. **Design the terminal interface.** Spectre.Console screens: title, character sheet, battle view with HP bars, shop and save confirmation.

- *Done when:* every screen from step 5 has its styled version.